On-Device AI 실습
KV Cache Pruning

Dongkun Shin
Intelligent Embedded Systems Lab.
Sungkyunkwan University

실습 목차

1. **KV Cache 메모리 계산**
   - Model의 Config로부터 캐시 바이트 수 계산, weight와 비교
2. **H2O [NeurIPS ’23]**
   - 누적 attention 기반 KV cache pruning 기법
3. **SnapKV [NeurIPS ’24]**
   - Observation window 기반 KV cache pruning 기법

Why Decoding is Memory-Bound

- **Decoding은 한 번에 토큰 하나만 생성**
  - 토큰 하나를 만들기 위해 weight 행렬 전체를 읽어야 함
  - Batch size가 1이면 모든 matrix multiply가 GEMV가 됨
- **Arithmetic intensity가 하드웨어 비율보다 훨씬 낮음**
  - FLOP/Byte of GEMV << (peak FLOPS) / (memory bandwidth)
  - Compute units idle
  - memory bandwidth is the bottleneck



해결 방안

- 메모리 읽는 양을 줄인다 → KV Cache Pruning
- 읽어온 김에 연산을 더 한다 → Speculative Decoding

Overview

KV Cache Pruning, Speculative Decoding을 통해 모델 크기는 그대로 두고, 추론시간의 메모리와 지연을 줄인다

KV Cache Pruning

– 긴 문맥에서 선형으로 커지는 KV Cache를 중요한 토큰의 Key/Value만 예산 안에 남김
– StreamingLLM · H2O · SnapKV를 동일한 scoring 인터페이스로 구현
– 정확도(passkey retrieval) vs 캐시 메모리를 비교

Speculative Decoding

– 작은 draft 모델이 여러 토큰을 미리 제안
– target 모델이 한 번의 forward로 검증
– chain → tree → parallel block으로 drafting 방식을 확장
– 출력은 AR decoding과 토큰 단위로 동일

두 기법 모두 memory-bound인 decoding 단계를 타겟하여 최적화를 진행함

PART 1

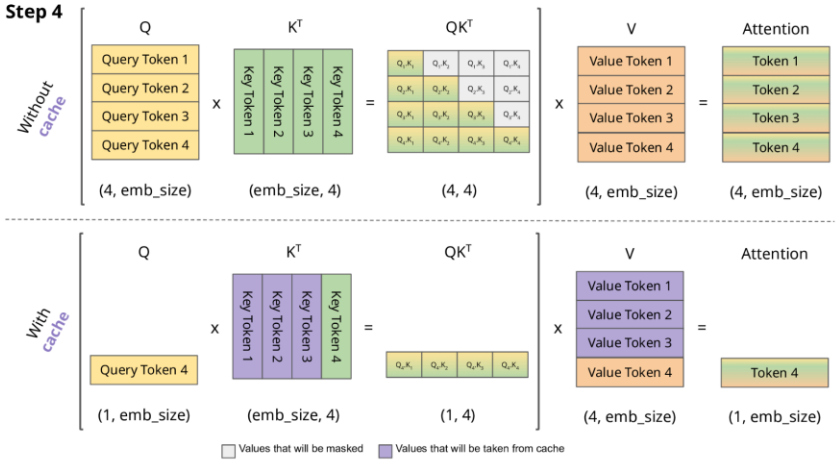
KV Cache의 구조와 메모리 병목

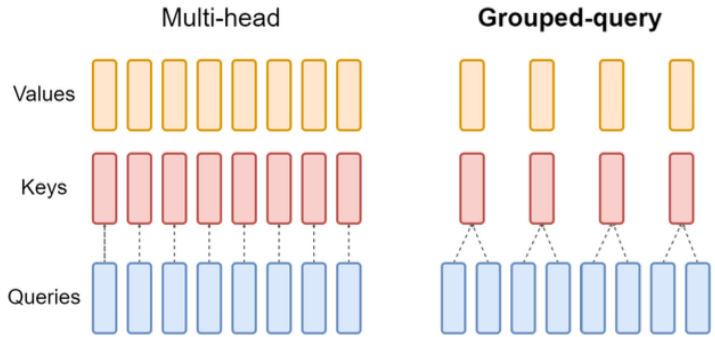
왜 decoding이 memory-bound이고, 캐시가 얼마나 커지는가

What is the KV Cache?

- **이미 계산한 Key/Value를 저장해 재계산을 없앤다**
  - 캐시가 없으면 매 step 마다 전체 시퀀스의 K/V를 다시 계산
  - 캐시가 있으면 새 토큰의 K/V만 계산하고 이어 붙임

- **캐시 크기가 문맥 길이에 정비례해 커짐**
  - Layer마다, KV head 마다 따로 저장
  - GQA는 KV head 수를 줄여 캐시를 크게 절감
    - Ex) Qwen2.5-1.5B
      12 query heads but only 2 KV heads → 6x smaller cache





https://medium.com/@joaolages/kv-caching-explained-276520203249

[실습 1] KV Cache 메모리 계산


```python
def kv_cache_bytes(config, seq_len, batch_size=1, dtype_bytes=2):
    """
    KV Cache가 차지하는 총 바이트 수를 계산합니다.

    Args:
        config     : transformers의 PretrainedConfig
        seq_len    : 문맥 길이 (토큰 수)
        batch_size : 배치 크기
        dtype_bytes: 원소 하나당 바이트 (FP32 -> 4, FP16/BF16 -> 2)

    Returns:
        int: 총 바이트 수
    """
    nconfig = model.config
    n_layers = config.num_hidden_layers
    n_q_heads = config.num_attention_heads
    n_kv_heads = config.num_key_value_heads
    head_dim = getattr(config,"head_dim", config.hidden_size // config.num_attention_heads)

    # ===== YOUR CODE STARTS HERE =====

    total_bytes =

    # ===== YOUR CODE ENDS HERE =====

    return total_bytes
```

[실습 1] Answer


```python
def kv_cache_bytes(config, seq_len, batch_size=1, dtype_bytes=2):
    """
    KV Cache가 차지하는 총 바이트 수를 계산합니다.

    Args:
        config     : transformers의 PretrainedConfig
        seq_len    : 문맥 길이 (토큰 수)
        batch_size : 배치 크기
        dtype_bytes: 원소 하나당 바이트 (FP32 -> 4, FP16/BF16 -> 2)

    Returns:
        int: 총 바이트 수
    """
    nconfig = model.config
    n_layers = config.num_hidden_layers
    n_q_heads = config.num_attention_heads
    n_kv_heads = config.num_key_value_heads
    head_dim = getattr(config,"head_dim", config.hidden_size // config.num_attention_heads)

    # ===== YOUR CODE STARTS HERE =====

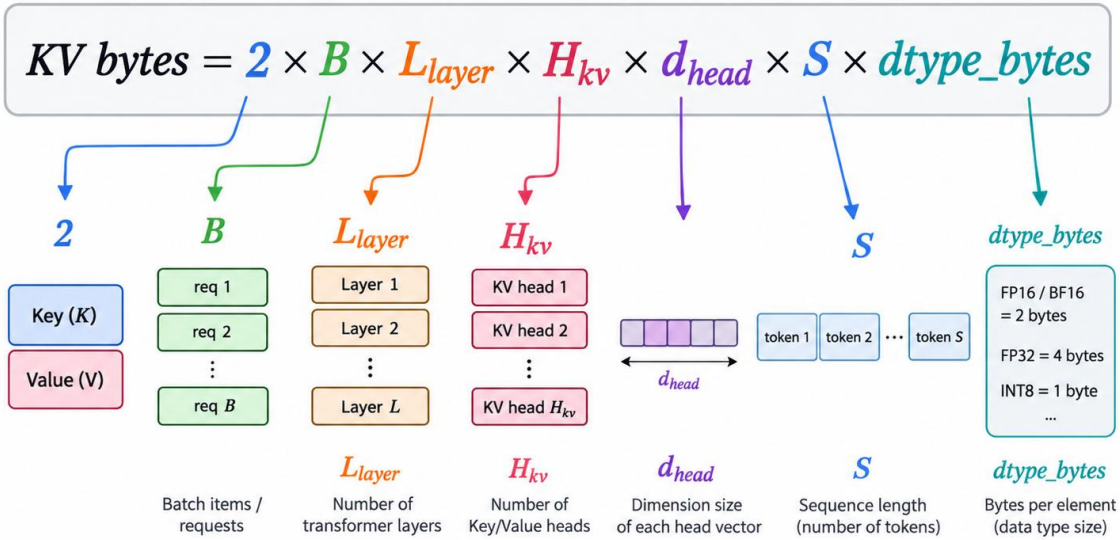
    total_bytes = 2 * batch_size * n_layers * n_kv_heads * seq_len * head_dim * dtype_bytes

    # ===== YOUR CODE ENDS HERE =====

    return total_bytes
```

[실습 1] KV Cache 메모리 계산

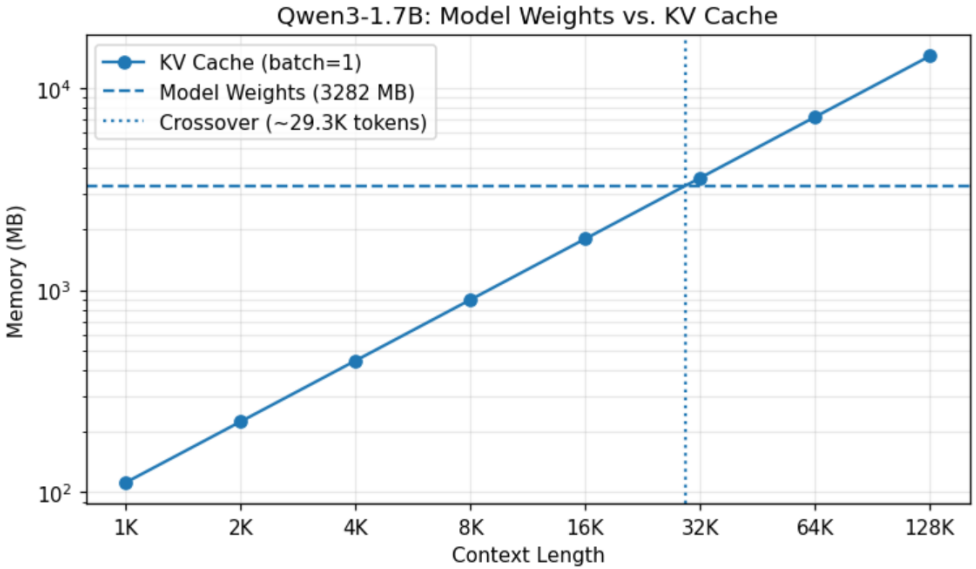
- **Implement kv_cache_bytes(config, seq_len, batch_size, dtype_bytes)**
  - $KV\ bytes = 2 \times B \times L_{layer} \times H_{kv} \times d_{head} \times S \times dtype_{bytes}$
  - The leading 2 counts Key and Value
  - H_{kv} is the number of KEY/VALUE heads, not query heads



[실습 1] KV Cache 메모리 계산

Qwen3-1.7B · FP16 에서의 관찰

- 문맥 길이에 정확히 **선형으로 증가**
- 어느 지점부터 KV Cache가 model **weight보다 커짐**
- 그 crossover 이후에는 weight 양자화보다 캐시 읽기가 더 큰 병목
- Batch size가 클수록, 문맥 길이가 길수록, KV Cache 문제가 커짐




```
Model weights     : 3281.7 MB
KV cache / token  : 112.0 KB
Crossover         : ~29.3K tokens
```

PART 2

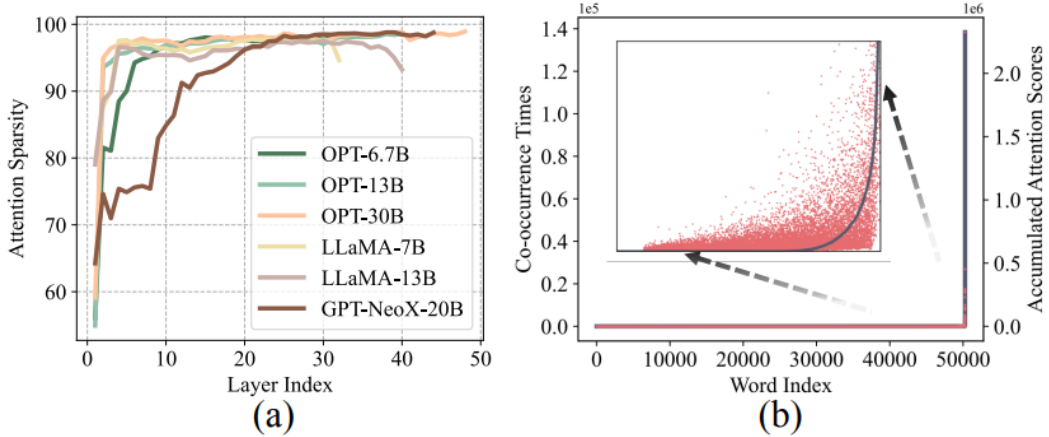
KV Cache Pruning

무엇을 남길 것인가 — StreamingLLM · H2O · SnapKV

Attention is Sparse

- **Attention은 매우 희소하고, 토큰 중요도는 heavy-tailed**
  - 각 decoding step에서 극히 일부 과거 토큰만 큰 attention을 받는다
  - 누적 attention score는 power-law 분포를 보인다

- **This creates an opportunity for KV-cache compression**
  - 많은 KV entry를 버려도 품질을 유지할 수 있다
  - The key challenge : which KV entries should be retained?



KV Cache Pruning: Two Design Axes

Scoring : 무엇이 중요한가

- 위치만 사용 → StreamingLLM
- 누적 Attention Score → H2O
- 누적 Attention Score + observation window → SnapKV

Budget : 얼마나 남길 것인가

- layer 균등 → H2O, SnapKV
- layer별 피라미드 → PyramidKV
- head별 적응적 배분 → Ada-KV, HeadKV

StreamingLLM: Attention Sinks [ICLR'24]

- **A naive sliding window collapses**
  - 맨 앞 토큰 몇 개가 윈도우를 벗어나는 순간 perplexity가 폭증
- **Cause: attention sinks**
  - softmax는 가중치 합을 1로 맞춰야 하므로 남는 가중치를 어딘가에 쏟는다
  - 맨 앞 토큰들이 그 잉여를 흡수하는 sink 역할을 한다
  - 초기 4개 토큰만 유지해도 잃었던 품질이 거의 회복된다
- **가장 값싼 정책**
  - Keeps the first n_sink tokens + a recent window, ignoring attention entirely



한계

- 중간 문맥을 통째로 버린다
- passkey가 앞이나 중간에 있으면 실패
- "최근 문맥만 중요한" 스트리밍 대화에 적합

StreamingLLM Score (제공 예시)


```
def streaming_score(attn_kv, layer, budget, n_sink=4):
    B, H, K, _ = layer.keys.shape

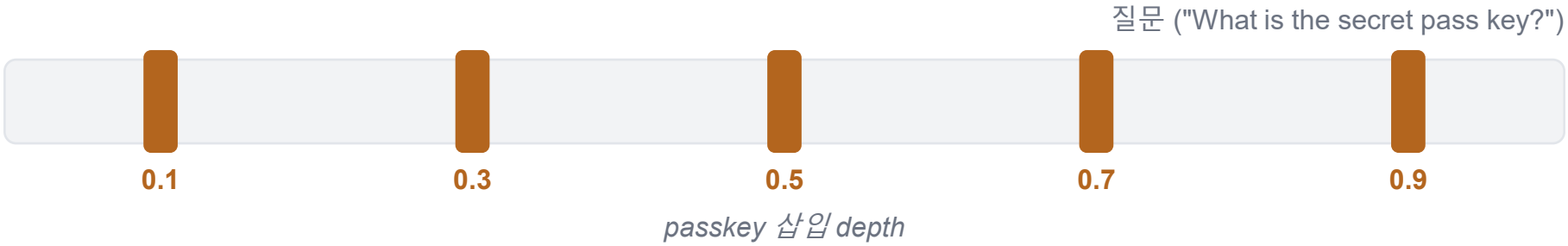
    # 최근 token일수록 높은 점수: 0, 1, 2, ..., K-1
    scores = torch.arange(K, device=DEVICE).float()
    scores = scores.repeat(B, H, 1)

    # 처음 n_sink개 token은 무조건 유지
    scores[:, :, :n_sink] = float("inf")
    return scores
```

- **attn_kv를 아예 사용하지 않는다**
  - attention 계산이 필요 없어 가장 값싼 정책
- **위치 자체가 score**
  - topk를 하면 자연스럽게 최근 window가 선택된다
  - sink 토큰은 inf로 강제 보존
- **이후 두 방법의 baseline 역할**

Evaluation: Passkey Retrieval

- **긴 filler 텍스트 안에 5자리 key를 숨기고 질문**
  - depth 0.1 / 0.3 / 0.5 / 0.7 / 0.9 로 삽입 위치를 제어
  - 5개 위치 중 몇 개를 맞히는지를 통해 정확도를 계산




```
depth=0.1 | answer=17412 | 0 | The secret pass key is **17412**
depth=0.3 | answer=17412 | 0 | The secret pass key is **17412**
depth=0.5 | answer=17412 | 0 | The secret pass key is **17412**
depth=0.7 | answer=17412 | 0 | The secret pass key is **17412**
depth=0.9 | answer=17412 | 0 | The secret pass key is **17412**
```

Full KV Cache에서의 결과 예시

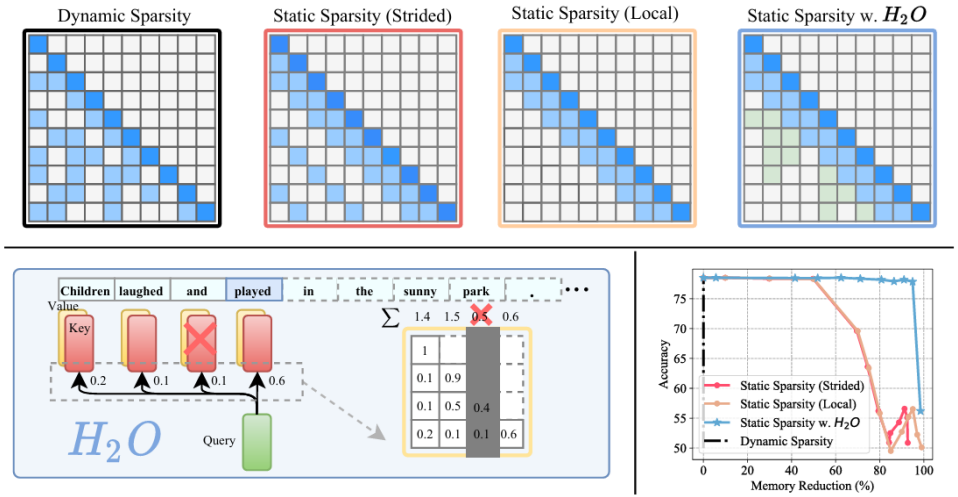
StreamingLLM 결과


```
depth=0.1 | answer=17412 | X | The secret pass key is **12345**
depth=0.3 | answer=17412 | X | The secret pass key is **12345**
depth=0.5 | answer=17412 | X | The secret pass key is **12345**
depth=0.7 | answer=17412 | X | The secret pass key is **12345**
depth=0.9 | answer=17412 | O | The secret pass key is **17412**
```



중간에 Evicted 되는 부분에
PassKey가 있을 경우 맞힐 수 없음

H2O: Heavy-Hitter Oracle [NeurIPS '23]



- **Keep the tokens that accumulate the most attention over all queries**
  - $score(j) = \sum \mathrm{A[i][j]}$
  - Combined with a recent window, as in every eviction method

- **Known weakness: recency bias from the causal mask**
  - Token j is only visible to N-j queries, so a plain sum favors early tokens

H2O 동작

① **추가**
새 토큰의 K/V를 캐시에 넣는다

② **갱신**
각 토큰의 누적 attention score를 더한다

③ **축출**
예산을 넘으면 score 최소 토큰을 버린다

- **예산 k는 두 몫으로 나눔**
  - heavy hitter가 절반, 최근 토큰(recent window)이 절반
  - recent가 없으면 방금 만든 문맥이 끊김
- **로컬 통계만으로 동작**
  - 미래 attention 없이, 현재 캐시에 남은 token의 누적 attention만 추적
  - 축출된 token은 이후 attention 계산 및 importance 갱신에서 제외

캐시 예산 k의 구성



본 실습의 단순화

step마다 축출하는 대신, prefill이 끝난 뒤 한 번만 pruning한다 (세 기법 비교를 위해 조건을 통일)

[실습 2] H2O 구현

- **Implement H2OPress.score()**
  - Reduce the query axis (dim=-2) of the attention weights
  - Keep the key axis intact
- **Then compare H2O and H2O-norm on passkey retrieval**

2-5. H2O Score

```
############실습코드 시작##############
# 전체 query의 attention 누적
scores = _________________________

# 최근 recent_size개 token은 항상 보존
scores[..., -recent_size:] = float("inf")
############실습코드 끝###############
```

[실습 2] Answer


```python
def h2o_score(attn_kv, layer, budget):

    recent_size = budget//2
    ############실습코드 시작##############
    # 전체 query의 attention 누적
    scores = attn_kv.sum(dim=-2)

    # 최근 recent_size개 token은 항상 보존
    scores[..., -recent_size:] = float("inf")
    ############실습코드 끝###############

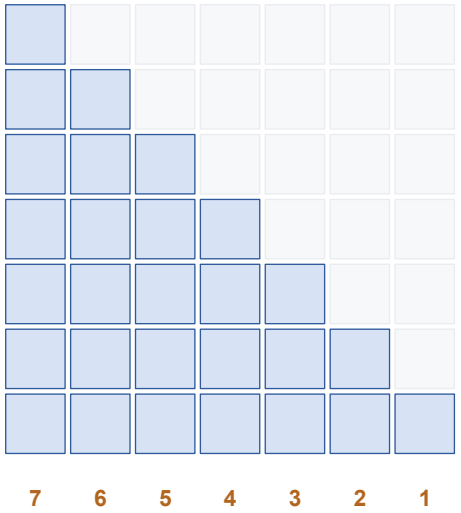
    return scores
```

```
depth=0.1 | answer=17412 | output=The secret pass key is **17412** | O
depth=0.3 | answer=17412 | output=The secret pass key is **17412** | O
depth=0.5 | answer=17412 | output=The secret pass key is **17412** | O
depth=0.7 | answer=17412 | output=The secret pass key is **12345** | X
depth=0.9 | answer=17412 | output=The secret pass key is **12345** | X
```

H2O의 구조적 편향

- **causal mask 때문에 앞쪽 토큰일수록 더 많은 query에서 합산된다**
  - 위치 j는 K-j개의 query에게만 보인다
  - 단순 합산은 앞쪽 토큰에 유리하게 왜곡된다 (원 논문도 지적)

- **H2O-norm: 실제로 attend된 query 개수로 나눈다**



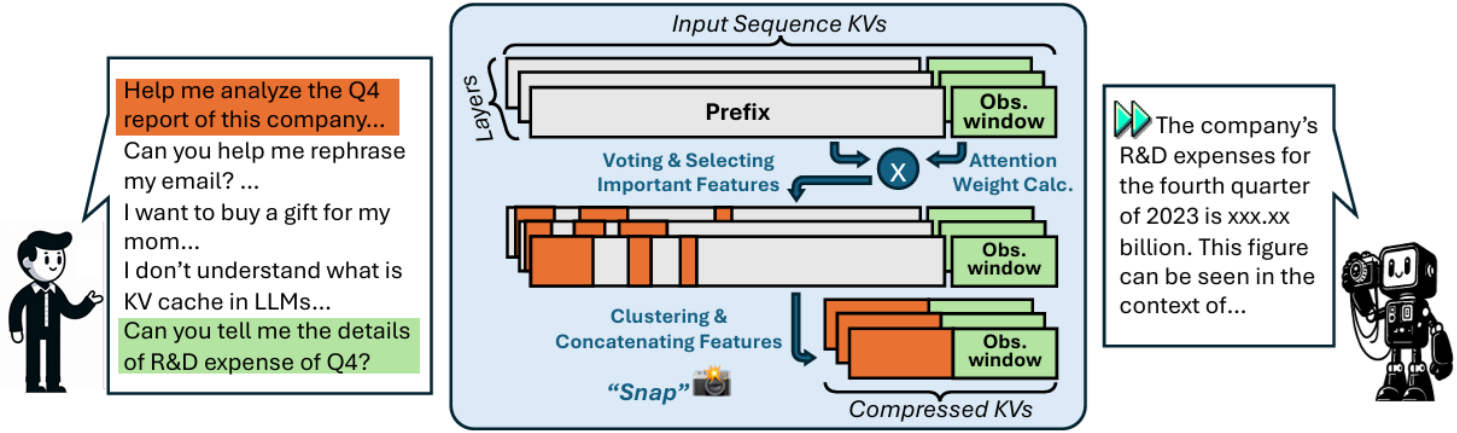
각 열이 실제로 받은 query 수

```
count = torch.arange(K, 0, -1, device=..., dtype=...)
scores[..., -recent_size:] = float("inf")
return scores / count.view(1, 1, K)
```

```
depth=0.1 | answer=17412 | output=The secret pass key is **17412** | O
depth=0.3 | answer=17412 | output=The secret pass key is **17412** | O
depth=0.5 | answer=17412 | output=The secret pass key is **17412** | O
depth=0.7 | answer=17412 | output=The secret pass key is **17411** | X
depth=0.9 | answer=17412 | output=The secret pass key is **17412** | O
```

정규화 유무를 같은 budget에서 비교

SnapKV: Observation Window + Pooling [NeurIPS '24]



- **Idea 1 - Observation window**
  - What the model will look at during generation is already visible in the attention of the last ~32 query positions of the prompt
  - Using only that window also removes H2O's recency bias

- **Idea 2 - Pooling clustering**
  - Raw top-k picks scattered, isolated tokens: broken fragments of text
  - 1D max pooling over the score makes neighbours survive together, so whole phrases are preserved

SnapKV의 동작

① 프롬프트를 prefix와 observation window로 나눔

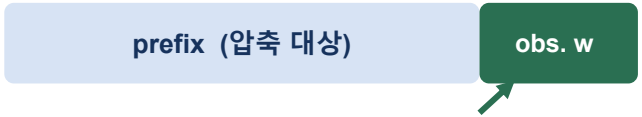
② window의 query가 prefix에 준 attention을 head별로 합산

③ 1D max pooling으로 점수를 클러스터링

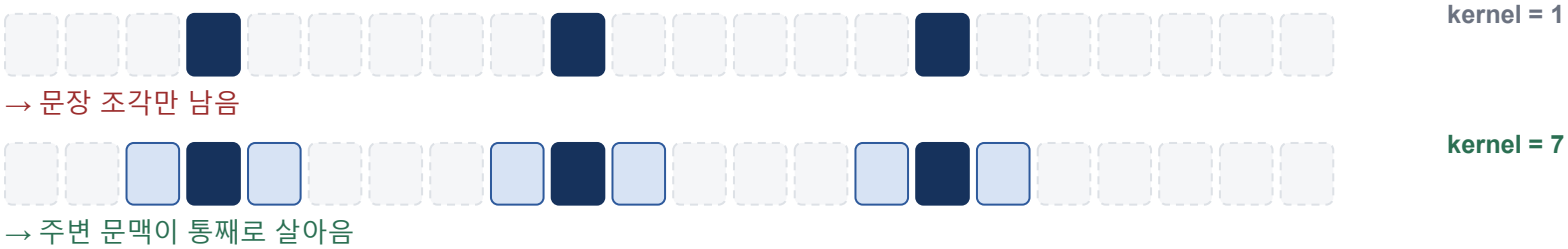
④ head마다 top-k를 골라 prefix KV를 압축

- **Observation window — 전체 query가 아니라 마지막 w개 query만 사용**
  - 생성 시점에 모델이 볼 곳을 프롬프트 끝부분이 미리 보여준다
  - 합산 대상 query 수가 모든 열에서 동일 → H2O의 recency 편향이 사라진다

프롬프트의 분할



- **Pooling clustering — raw top-k는 흩어진 고립 토큰을 고름**



[실습 3] SnapKV 구현

- Step 1: aggregate attention over the last w queries only
- Step 2: apply max pooling(kernel_size=k, padding=k//2, stride=1)

2-7. SnapKV Score

```
########################실습코드 시작#############################
    # TODO 1
    # 마지막 w개의 query가 각 KV token에 준 attention을 합산하세요.
    # Hint: attn_kv shape = [B, H, Q, K]
    scores =_____________________
    …
    for j in range(K):
        …
        # TODO 2
        # Hint: start ~ end 위치 중 최대 score
        pooled_scores[:, :, j] = ____________________
        …
########################실습코드 끝#############################
```

[실습 3] Answer


```python
def snapkv_score(attn_kv, layer, budget, window=32, kernel=7):
    B, H, Q, K = attn_kv.shape
    w = min(window, Q)

    ########################실습코드 시작#########################

    # TODO 1
    # 마지막 w개의 query가 각 KV token에 준 attention을 합산하세요.
    # Hint: attn_kv shape = [B, H, Q, K]
    scores = attn_kv[:, :, -w:, :].sum(dim=2)

    # 각 token 주변 kernel 범위에서 가장 큰 score를 가져옵니다.
    half = kernel // 2
    pooled_scores = scores.clone()

    for j in range(K):
        start = max(0, j - half)
        end   = min(K, j + half + 1)

        # TODO 2
        # Hint: start ~ end 위치 중 최대 score
        pooled_scores[:, :, j] = scores[:, :, start:end].max(dim=-1).values

    scores = pooled_scores

    # Observation window는 항상 유지
    scores[:, :, -w:] = float("inf")
    ########################실습코드 끝#########################

    return scores
```

```
depth=0.1 | answer=17412 | output=The secret pass key is **17412** | 0
depth=0.3 | answer=17412 | output=The secret pass key is **17412** | 0
depth=0.5 | answer=17412 | output=The secret pass key is **17412** | 0
depth=0.7 | answer=17412 | output=The secret pass key is **17412** | 0
depth=0.9 | answer=17412 | output=The secret pass key is **17412** | 0
```

통합 비교: 정확도 vs KV Cache 메모리

StreamingLLM

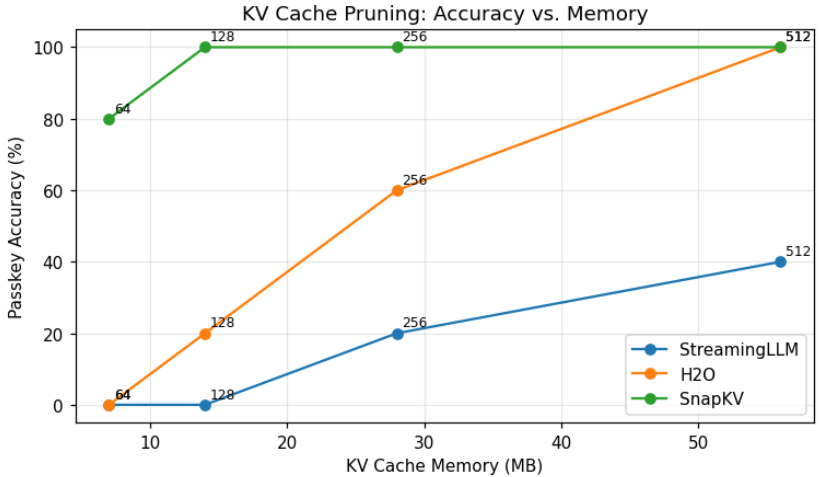
- 위치 기반 (attention 불필요)
- 가장 값쌈 — prefill attention 없음
- 앞·중간 passkey에서 실패
- 최근 문맥만 중요한 스트리밍에 적합

H2O

- 전체 query 누적 attention
- 중간의 중요 토큰도 포착
- causal mask에 의한 recency 편향
- budget을 줄이면 먼저 무너짐

SnapKV

- 마지막 w개 query + max pooling
- 편향 없음 + 문맥 덩어리 보존
- window 크기·kernel에 민감
- 같은 예산에서 가장 안정적



Part 2 정리

eviction 계열의 근본 한계

한 번 버린 토큰은 되살릴 수 없다. 프롬프트를 볼 때 중요하지 않던 토큰이 뒤에서 필요해지면 방법이 없다.

다른 KV cache 최적화 연구 방향들

KV 양자화

`KIVI`
`TurboQuant`

버리지 않고 비트 수를 줄임

저랭크 분해

`Palu`

KV를 저차원으로 투영해 저장

Sparse attention

`Quest`
`Block Attention`

전체 캐시는 유지하고 일부만 읽는다

예산 배분

`PyramidKV`
`Ada-KV`

layer·head별로 budget을 다르게 설정